

從零到全端上線：現代 Web 應用開發完整教學手冊

以「三人好友共學自律與公費金庫系統」為實例，解析現代網站的誕生全流程

實戰專案：self-discipline

技術棧：React + Vite + Supabase + Cloudflare

日期：2026 年 9 月

💡 給大一新生的生動比喻：現代網站到底是什麼？

很多初學者以為寫網站就是單純寫 HTML，但一個完整上線的網路服務就像開一家「無人智慧咖啡館」：

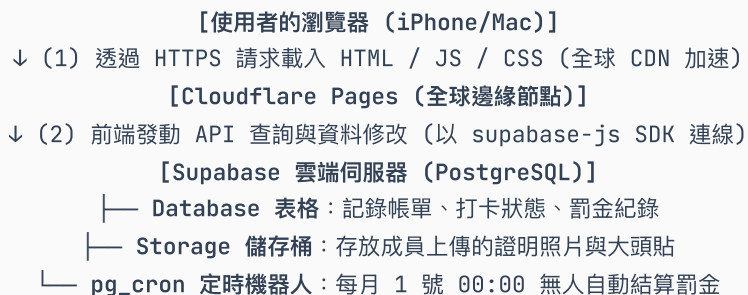
- **前端 (Frontend, React + Tailwind)**：店裡的裝潢、點餐觸控螢幕、菜單翻頁與按鈕回饋。你眼睛看得見、手指摸得到的所有東西。
- **後端與資料庫 (Backend, Supabase)**：後台的倉庫與帳本。記錄「誰點了什麼餐」、「倉庫還剩多少豆子」、「誰還沒付錢」，沒有它點餐機關機後資料就全忘光了。
- **雲端託管 (Deployment, Cloudflare Pages)**：將這家店蓋在全世界最高速的交通樞紐上，不管是誰從手機或電腦輸入網址，都能在一瞬間進店參觀。

目錄大綱

章節	核心內容	學習收穫
第 1 講：全端架構宏觀視野	前後端分離、資料庫的角色、邊緣網路概念	搞懂「按一個按鈕到網頁更新」中間經歷了什麼
第 2 講：本機開發環境搭建	Node.js、npm、Vite 現代化建置工具	學會在自己的電腦終端機啟動現代前端伺服器
第 3 講：前端核心架構開發	React 組件化、State 狀態驅動、iOS 液態玻璃設計	掌握如何將一個複雜的大介面拆解成小樂高積木
第 4 講：雲端資料庫與儲存庫	Supabase (PostgreSQL)、雲端照片儲存、pg_cron 排程	學會建立能自動跨日刷新、跨月結算的雲端後端
第 5 講：版本控制與協作	Git 時光機、.gitignore 避坑、SSH 密鑰設定	掌握工程師必備的代碼保存與 GitHub 倉庫管理
第 6 講：全球邊緣部署上線	Cloudflare Pages 連動、環境變數保密、防快取策略	把本地代碼變成全世界都能點開的真實網址
第 7 講：實戰避坑與排錯思維	建置指令錯誤排查、瀏覽器強快取解析、開發心法	培養面對 Bug 臨危不亂的資深工程師思維

第 1 講：全端架構宏觀視野

在開始敲代碼前，讓我們先拉高視角，看懂這次專案的整體資訊流動架構：



過去傳統作法需要自己買一台 Linux 雲端主機（例如 AWS EC2），自己安裝作業系統、設定 Nginx、安裝 Node.js、設定防火牆。而現在 2026 年的現代全端開發流行 **Serverless (無伺服器架構)** 與 **BaaS (後端即服務)**，我們只需專注於寫好商業邏輯，基礎設施全交給頂尖雲端平台維運！

第 2 講：本機開發環境搭建

1. 認識 Node.js 與 npm

瀏覽器本來只能在網頁裡執行 JavaScript。但 **Node.js** 把它搬到了電腦本機，讓工程師可以在終端機裡執行 JS，甚至編譯代碼、安裝開源工具。而 **npm (Node Package Manager)** 就是世界最大的 JavaScript 開源軟體超級市場。

🔧 必備指令清單

```
# 檢查本機 Node.js 與 npm 版本
node -v      # 顯示如 v20.18.0
npm -v       # 顯示如 10.8.2

# 將自訂安裝路徑加入 macOS 終端機環境變數 (~/.zshrc)
export PATH="$HOME/nodejs/bin:$PATH"
```

2. 使用 Vite 快速建立現代前端骨架

過去前端工具（如 Webpack）啟動非常慢，改一行代碼要等好幾秒。而 **Vite** 利用了現代瀏覽器的原生 ES 模組支援，啟動速度只需 **0.2 秒**，熱更新（Hot Module Replacement）完全無感！

```
# 透過 Vite 腳本在當前目錄建立 React + TypeScript 專案
npx -y create-vite@latest ./ --template react-ts

# 安裝專案核心套件
npm install @supabase/supabase-js framer-motion lucide-react canvas-confetti date-fns
npm install -D tailwindcss postcss autoprefixer
```

第 3 講：前端核心架構與組件化設計

1. 為什麼一定要學 React 組件化 (Component-Based)？

如果你用純 HTML 寫 10 個相似的任務卡片，你就得複製貼上 10 次整段 HTML，只要改一個按鈕顏色就得改 10 處！

而 React 就像**樂高積木**，我們把一個任務卡片封裝成 `TaskCard` 組件，傳入不同的資料（如標題、是否完成），它就自動渲染出不同的樣貌：

🧩 本專案的前端組件劃分

- `App.tsx`：總指揮官，管理全局狀態（當前登入者、任務清單、金庫金額、右上角頭像秒切換）與定時器。
- `AuthModal.tsx`：操作者身分快速切換面板，點擊右上角頭像隨時彈出秒切換帳號。
- `BottomNav.tsx`：底部 iOS 懸浮液態玻璃導覽列，支援滑動動態指示膠囊。
- `HomeTerminal.tsx`：個人自律終端，負責常駐每日必做、今日任務與明日預排。
- `TreasuryView.tsx`：公費金庫儀表板，支援修改夢想標竿與瀏覽每月 1 號自動歸檔帳單。
- `WeeklyMatrix.tsx`：週一至週日里程碑排程矩陣。
- `SettingsView.tsx`：個人專屬設定，支援直接從本機上傳大頭貼。

2. 打造極致 iOS 液態玻璃 (Liquid Glass) 風格

這次我們徹底捨棄俗氣的純黑純白，採用類似 Apple VisionOS / macOS 的黑白極簡磨砂玻璃：

- **背景層**：採用極黑（`#000000`）搭配微弱的白色徑向光暈（`radial-gradient`）。
- **卡片層**：採用半透明微白（`bg-white/[0.03]`）搭配高強度背景模糊（`backdrop-blur-2xl`）與極細緻半透明邊框（`border-white/10`）。
- **懸浮 Dock**：底部導覽列做成圓角膠囊（`rounded-full`），浮在內容上方，搭配流暢的彈簧物理動畫（`framer-motion`）。

第 4 講：雲端資料庫與儲存庫 (Supabase BaaS)

一個單純的前端網頁，重新整理之後輸入的資料就會全消失。為了讓資料永久留存並在三人之間同步，我們引入了 **Supabase**（基於最強關聯式資料庫 PostgreSQL 的雲端服務）。

1. 資料庫綱要設計 (Schema Design)

資料表名稱	主要欄位	用途說明
profiles	id (UUID), username, avatar_url, total_paid_fine	存放 Johnson、nigga、shorty 的身分、頭像網址與歷史累積罰金
group_settings	goal_name, goal_amount, cover_photo	存放三人的共同夢想標竿（如古民家買房基金與金額）
daily_routines	id, user_id, title	常駐每日必做模板表（永久保存核心習慣項目，隨時可增修）
tasks	user_id, routine_id, title, category, is_completed, proof_url	每天具體的打卡實例（記錄照片佐證與豁免理由）
monthly_bills	user_id, billing_month, failed_tasks_count, fine_amount	每月 1 號由系統自動結算產出的月結正式歸檔帳單

2. 雲端圖片儲存桶 (Supabase Storage)

資料庫裡面只適合放文字與數字，千萬不能直接把幾 MB 的照片存進資料庫欄位！正確做法是：

- 前端透過 `supabase.storage.from('task-proofs').upload(...)` 將照片二進位檔上傳到雲端儲存桶。
- 儲存桶回傳一個全世界都可訪問的 CDN 圖片網址（如 `https://xxx.supabase.co/storage/v1/object/public/task-proofs/abc.jpg`）。
- 最後只把這串「網址字串」寫入 `tasks.proof_url` 或 `profiles.avatar_url`！

3. 雙軌自動化月結算歸檔 (pg_cron + 客戶端自動檢查)

🤖 雲端與客戶端雙軌歸檔：雙保險保證每月 1 號自動歸檔！

為了確保每位成員的月結帳單在每月 1 號 00:00 必定歸檔，我們設計了現代全端架構的雙軌保險機制：

- **軌道一（伺服器排程）**：在 Supabase 資料庫撰寫 `monthly_settlement_audit()` 預存程序，透過 `pg_cron` 設定每天台灣午夜檢查，只要逢每月 1 號自動喚醒執行。
- **軌道二（前端自動補償）**：當成員於每月 1 號（或次月任何時間）打開網頁時，`checkAndAutoArchiveMonthlyBills()` 會自動核對上月是否已歸檔，若尚未歸檔立即自動結算並寫入資料庫！
- **防重複扣款保護 (Idempotency)**：資料庫設有 `(user_id, billing_month)` 唯一約束，同一月份絕對不重複產生帳單。

第 5 講：版本控制與團隊協作 (Git & GitHub)

工程師寫程式最怕「改壞了回不去」或是「電腦壞了代碼全滅」。Git 是代碼的時光機，而 GitHub 則是代碼在雲端的雲端硬碟兼協作平台。

1. 關鍵防雷神器：.gitignore

⚠ 新手最常犯的重大致命錯誤！

新手常不小心把 `node_modules/`（幾萬個龐大檔案）或 `.env`（裡面有你的私密金鑰！）全部推上 GitHub。公開的 GitHub 倉庫有機器人 24 小時巡邏，若金鑰外洩可能導致資料庫被駭！因此必須在專案根目錄建立 `.gitignore`，明確告訴 Git 忽略這些檔案！

2. 現代 GitHub 推送必備：SSH 密鑰 (SSH Key)

自 2021 年起，GitHub 為了安全性已經全面禁止在終端機輸入帳號密碼來 push 代碼。我們必須在本機產生一組非對稱加密密鑰：

```
# 1. 產生高強度的 ed25519 SSH 金鑰
ssh-keygen -t ed25519 -C "你的GitHub帳號" -f ~/.ssh/id_ed25519 -N ""

# 2. 將公鑰複製到剪貼簿並貼到 GitHub Settings → SSH Keys
cat ~/.ssh/id_ed25519.pub | pbcopy

# 3. 關聯遠端倉庫並安全推送
git remote set-url origin git@github.com:Johnson-bro-coder/self-discipline.git
git push -u origin main
```

第 6 講：全球邊緣部署上線 (Cloudflare Pages)

寫好的代碼只跑在自己的 `localhost:5173` 不叫網站。要讓所有人透過網址打開，我們選擇了全世界頂尖的 CDN 服務商 **Cloudflare Pages**。

1. 現代化的 GitOps 自動化持續交付 (CI/CD)

當你在 Cloudflare Pages 連結 GitHub 倉庫後，就啟動了現代化的 **CI/CD (Continuous Integration / Continuous Deployment)** 流水線：

你在本地修改代碼 → `git commit` → `git push origin main`
↓ (GitHub 收到推送，發送信號給 Cloudflare)
Cloudflare 伺服器自動拉取最新代碼 → 執行 `npm run build` → 產出 `dist/`
↓ (將編譯後的靜態檔案發布至全球 300+ 邊緣節點)
🚀 約 1 分鐘內，全世界上線網址自動完成更新！

2. 環境變數 (Environment Variables) 的藝術

在本地我們寫在 `.env` 檔案裡，但在上線伺服器，我們要在 Cloudflare Pages 後台設定：

- `VITE_SUPABASE_URL`：你的 Supabase 專案網址
- `VITE_SUPABASE_ANON_KEY`：你的 Supabase 公開 anon 金鑰
- `VITE_PENALTY_UNIT`：違規罰金單位 (100)

3. 單頁應用 (SPA) 路由與防止瀏覽器強快取策略

Vite 構建的是 SPA (Single Page Application)，所有的頁面切換都是前端 JS 模擬的。

為了解決「重新整理出現 404」與「每次部署後使用者瀏覽器卡在舊版」的兩大難題，我們在 `public/_headers` 加入了精密快取控制：

```
# public/_headers
/*
  Cache-Control: no-cache, no-store, must-revalidate

/assets/*
  Cache-Control: public, max-age=31536000, immutable
```

策略精髓：對入口 `index.html` 永遠不快取（保證第一時間拿到最新代碼指標）；對帶有雜湊字尾的靜態資源（如 `index-CdkHJmJQ.js`）開啟長效快取，兼顧了秒開速度與永遠即時更新！

第 7 講：實戰避坑指南與全端工程師思維

遭遇問題	根本原因	工程師排查解法
部署後顯示 <code>VitePress 404</code> <code>PAGE NOT FOUND</code>	Cloudflare Pages 預設範本選成了寫文檔用的「VitePress」而非「Vite」，自動跑了 <code>npx vitepress build</code> 。	至專案後台 Settings → Builds → 將 Framework Preset 改為 <code>Vite</code> ，Build command 改為 <code>npm run build</code> 。
執行 git push 跳出 <code>Password authentication is not supported</code>	GitHub 早已停止帳號密碼驗證，防止密碼被暴力破解。	改用 SSH Key 金鑰對認證，或改用 Personal Access Token (PAT)。
代碼推送後打開網頁沒看到更新	瀏覽器磁碟快取 (Disk Cache) 自作聰明讀取了舊的 HTML。	1. 使用者端按 <code>Cmd + Shift + R</code> 強制刷新。 2. 伺服器端配置 <code>no-cache</code> 標頭徹底杜絕。

📖 結語：寫給未來的資深工程師

恭喜你！走完這份手冊，你已經親手經歷了一個現代軟體工程師在矽谷或科技大廠最核心的工作流：

需求分析 → 架構拆解 → UI/UX 動態美學 → 資料庫與儲存桶設計 → 定時排程無人化 → 版本控制 → CI/CD 全球部署。

技術的語法會隨著年份更迭，但這種「拆解問題、分層解決、邊做邊修」的工程思維，將是你大學四年與未來工程師生涯最強大的核心競爭力！